In-Home Assistive Robot

Detailed System Design

Design D1

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

October 8, 2026

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI assisted with drafting and LaTeX formatting.

1 Header, scope, and conventions

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

This document details the **call server** and **person tracking**; the resident controls, call manager, audio/video calling, motion control, robot platform, and caregiver app are deferred to D2. We picked these two because the call server holds the data model and person tracking is the computation that breaks following if it is slow or wrong. The APIs in Section 5 also cover where they meet their neighbors (I1, I2, I7, I8, and I14). The caregiver app appears here only as the client of I1 and as a technology choice in Section 6.

Figure conventions. Figure 1 is the only figure. Each blue box is an entity, with its name in bold and its key attributes below it. Each solid line is a relationship; lines have no arrows because they are not data flow. PK marks a primary key, FK a foreign key, and UQ a field covered by a unique index. Cardinality is written at both ends of every line: **1** means exactly one and **0..*** means zero or more. The number at a line end tells how many of the entity next to it belong to one record at the other end; for example, the **0..*** next to Pairing means one robot can have zero or more pairings. The legend repeats this on the figure.

This is a proposed build plan for one resident, one caregiver, and one robot on a clear indoor route (US-01 through US-05). Existing work includes Python simulation/perception prototypes, a consent/following bridge, and JavaScript screen mocks. The call server, database, production sign-in, and iOS companion are proposed integrations, not deployed services. The caregiver app will request/end calls and display outcomes through I1, use I13 for sign-in, and use I12 for media after resident acceptance; it cannot grant itself CALL or FOLLOW permission.

Evidence boundary. Real public-video experiments did not establish sustained selected-person tracking: the initial acquisition failed, and a later same-sample selection lasted only frames 5–7 before loss. Synthetic RGB-D integration suppressed travel requests but failed stationary hold. Some later bounded simulations improved nominal stopping; feedback-loss cases still failed. No physical stopping, calibrated live person-depth, or integrated mobile call has been demonstrated. Time limits are design targets, not measured deployment guarantees.

2 Data model, the D1 diagram

The call server uses a relational SQLite database. Figure 1 records account-to-robot enrollment and call outcomes. A pairing permits a caregiver's account to request a call; the resident's separate CALL and FOLLOW permissions remain on the robot and can still deny it (US-02, US-03; I1, I2, I14).

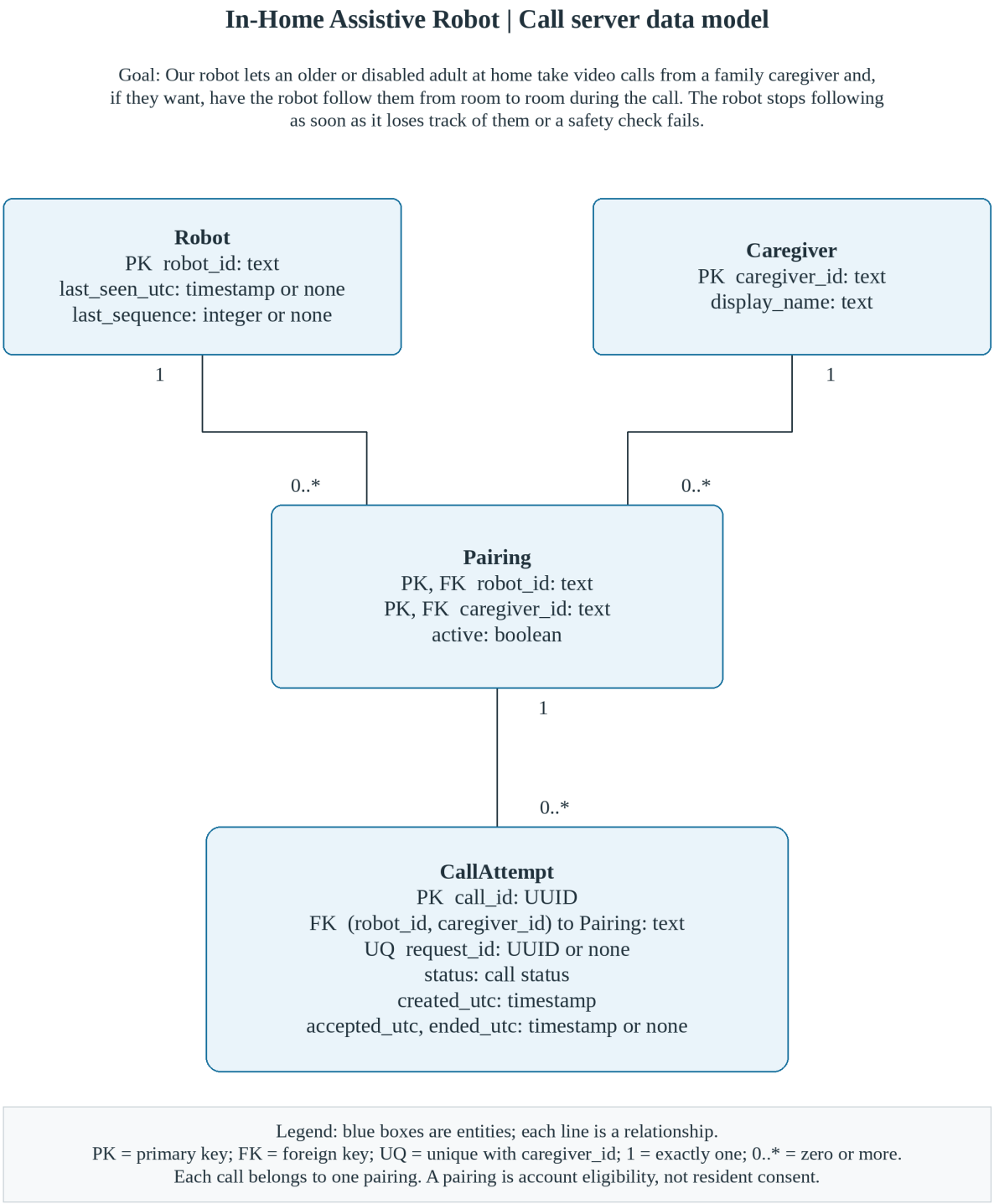


Figure 1. Call server data model

A robot and a caregiver can each have zero or more pairings; each pairing names exactly one of each. One pairing can have zero or more calls, and each call belongs to exactly one pairing through the composite foreign key (robot_id, caregiver_id). An inactive pairing (active = false) stays in the database so its old calls keep their owner. Robot stores last_seen_utc and last_sequence so the server can tell a fresh heartbeat from a repeated one (I2). We store no audio, video, passwords, access tokens, resident locations, or medical records. accepted_utc and ended_utc are null until those events happen, and all timestamps

are RFC 3339 UTC text. The status values are ringing, connected, declined, no_answer, offline, ended, and failed.

Table 1. Structural decisions and requirement links

Decision	Reason
CallAttempt is an entity	Each attempt needs its own time, status, and owner. A single status field on Robot would erase earlier attempts (US-03).
Pairing is an entity	A caregiver and a robot can each be associated with more than one counterpart. The separate pair records that many-to-many relationship and its active/revoked state; it does not replace resident consent (US-02).
Pairing to CallAttempt is one-to-many	One attempt belongs to one robot-caregiver pair; that pair can have many attempts. Sharing one attempt among several pairings would make its outcome ambiguous (US-01, US-03).
Heartbeat sequence is an attribute of Robot	Only the newest heartbeat matters, so we keep one last_sequence value instead of a Heartbeat table that would grow by one row per second. The algorithm compares against it to ignore repeated or out-of-order heartbeats (I2).
Keys and indexes	The Robot and Caregiver primary keys and the composite Pairing key support direct lookups. CallAttempt is indexed on (robot_id, created_utc) so the server can find a robot's latest call quickly. (caregiver_id, request_id) has a unique index so a retried start finds the original call instead of making a second one (I1). A partial unique index on robot_id for ringing or connected calls stops two active calls on one robot (US-03).
Relational store	We chose a relational model in SQLite because the rules above are constraints and transactions, which SQLite enforces for us on a single server. A plain JSON file would make simultaneous updates and uniqueness our problem. Only the call server opens the database file; the robot talks to it over I2.

3 Core algorithms

1. Call attempt and outcome handling

What it is and the problem it solves. The call server takes a caregiver's request and the robot's replies and turns them into one current outcome for each call. If it gets this wrong, the caregiver could see "connected" before the resident has accepted, or a decline could be overwritten by a later message. If it is slow, the caregiver waits on a call that already failed (US-02, US-03; I1, I2, I14).

Inputs and outputs. From the caregiver app (I1): robot_id: text, action: start or end, optional request_id: UUID, call_id: UUID (for end), and a Bearer access token in the request header. From the robot (I2): a heartbeat with sequence: 64-bit nonnegative integer, or an event with call_id: UUID and event: enum. Output: call_id: UUID, status: call-status enum, and last_seen: UTC timestamp or null. A rejected input returns an error code and message, both text. Exact ranges are in Section 5.

How it works. The server checks the token with the sign-in service and then checks for an active pairing. For a new start, it opens one database transaction, looks up the request_id if one was sent, checks the robot's last heartbeat, and inserts one CallAttempt. If the robot is offline the call is recorded as OFFLINE; otherwise it is RINGING and the server sends an invite over I2.

From RINGING, the resident's answer has to come through the robot. DECLINED is final. ACCEPTED records accepted_utc but the status stays RINGING until the robot reports MEDIA_CONNECTED; if that does not come within 2 seconds the call becomes FAILED. With no acceptance after 60 seconds the call becomes NO_ANSWER. The two-second acceptance-to-display target includes network and app time; the server timeout alone does not demonstrate it. A heartbeat only counts if its sequence is greater than last_sequence; if no counted heartbeat arrives for 5 seconds, the robot is reported OFFLINE with its last-seen time. ENDED, DECLINED, NO_ANSWER, OFFLINE, and FAILED are terminal, so reconnecting always means a new call and a new acceptance. Timeouts use the server's monotonic clock, and stored times are UTC for display. On server restart, a transaction closes stored ringing/connected attempts as FAILED with an end time. Every robot then counts as offline until it opens a new authenticated session and sends a heartbeat. A new session retires the old connection and resets its heartbeat sequence baseline, so a robot reboot can start at zero; events from the retired connection are rejected. Calls do not resume automatically.

Complexity at our data size. Each request does a constant number of indexed lookups and one insert or update, so the database work is O(log M) for M stored calls. A planning workload of one robot, one caregiver, and 10 calls a day retained for 30 days gives M = 300; the rate and retention period are assumptions to confirm. At 100 times that history (M = 30,000), log-base-2 M grows from about 9 to 15. These illustrate logarithmic growth, not SQLite page reads, measured comparisons, or latency. Network checks and write contention still need measurement. Heartbeats are the part that grows: 1 write per second for one robot becomes about 100 writes per second for 100 robots. SQLite allows one writer at a time [2], so that is the first thing we would benchmark before scaling up, and the reason PostgreSQL is our fallback in Section 6.

Why this approach. We compared three options. (1) Keeping only the latest status in a file or a Robot column is simpler, but it loses call history (US-03) and cannot settle two starts that arrive together. (2) Letting the robot own the call state and the server only relay messages fails when the robot goes offline: the caregiver would get no answer at all, which is exactly the case US-03 cares about. (3) A server-side state machine in a database transaction, with a partial unique index for one active call per robot, keeps history, handles the offline case, and lets the database reject races. We chose (3). We also push status over a secure WebSocket rather than having the app poll every second, which would add up to a second of delay and constant load for no new information.

Edge cases.

- *Empty or malformed input:* missing fields, bad UUIDs, or unknown fields return 400 INVALID_REQUEST and change nothing (I1).
- *Duplicates:* a repeated start with the same caregiver, request_id, and robot returns the original call. A repeated robot event or heartbeat does not change a terminal call or refresh last-seen time (I2).
- *Ties and races:* if two starts for one robot arrive together, the partial unique index lets one insert

win and the other gets 409 ACTIVE_CALL. If an acceptance arrives after the 60-second deadline, arrival time decides and the call stays NO_ANSWER.

- *Retry ID misuse:* a request_id reused for a different robot returns 409 REQUEST_CONFLICT.
- *Failures:* if the sign-in service or database is down the server returns 503 and never grants access by default (I14). A missing media confirmation or heartbeat never turns into CONNECTED.

2. Selected-person tracking and loss handling

What it is and the problem it solves. This runs on the robot. It keeps track of the one person the resident selected and gives motion control their distance and direction. If it is wrong, the robot can follow a visitor; if it is slow, the robot steers toward where the resident used to be. So it must stop giving a usable position whenever its data is stale, missing, ambiguous, or no longer allowed (US-01, US-02, US-04; I7, I8, I9).

Inputs and outputs. Inputs are the target ID, CALL/FOLLOW gates, permission generation, frame times, detector candidates, an optional selected-instance mask, and a calibrated depth-projection result for that same target, frame, and generation. Pixel positions and projected optical depth are finite floats; timestamps/generation are nonnegative 64-bit integers. Output is an I7 TrackRecord containing target ID, valid/unavailable status, range/bearing or null, capture/expiry times, a reason, coordinate frame, and a false motion-authorization flag. Section 5 defines the types. This is an offline candidate contract; conversion to a qualified robot-frame position remains integration work.

How it works. After accepting a call and pressing Follow, the resident picks one visible person on the screen. On each new frame:

1. *Gate.* Reject the frame if permission is off, its timestamp is in the future or more than 200 ms old, image/depth capture times differ by more than 50 ms, or calibration evidence is missing. The 200 ms freshness gate and 0.8 detection filter come from the existing prototype; the 50 ms skew and association/range thresholds below remain provisional design values.
2. *Associate.* Keep candidates with confidence at least 0.8. For a new explicit selection, use the named candidate without a prior-box match; reject overlapping ambiguous candidates. On later frames, compare intersection over union (IoU) with the last selected box. Accept a best IoU of at least 0.30 only if it beats the second best by more than 0.15. If another kept box overlaps the winner by 0.50 IoU or more, report ambiguity. These geometric tests do not establish identity assurance.
3. *Measure.* Require the exact selected-person mask and a candidate-status result from the calibrated projection adapter, bound to the selected target, frame, and generation. That adapter must check per-zone rays, radial range-error intervals, camera intrinsics/extrinsics, timing/pose uncertainty, and independent clear visibility. The whole projected zone envelope must lie inside the selected mask; a centre pixel or a boolean registration flag is insufficient. Reject inconsistent depth rather than selecting a convenient nearest cluster. Require at least three admitted zones whose entire depth intervals lie in the provisional 0.2–4.0 m domain. For offline inspection, range_m is median RGB optical-axis Z, distinct from raw radial ToF range and robot stopping clearance. Bearing uses arctan((mean projected x - cx_px) / fx_px) in degrees, positive right.

4. *Publish.* Recheck the permission generation and expiry at publication. Emit a VALID offline candidate only while fresh, with motion_authorized=false; otherwise report UNAVAILABLE. Current projection experiments do not grant movement authority.

If any step fails, publish UNAVAILABLE with null range and bearing, latch the loss, and wait for the resident to select and press Follow again. A watchdog publishes the loss even if no new frame arrives. Motion control must reject these motion-disconnected candidates. A future movement path also needs validated identity, a calibrated transform to the robot frame, independent clearance, freshness, resident permission, and qualified stopping. A zero travel request is not a demonstrated physical stop; the recorded feedback-loss failures remain unresolved.

Complexity at our data size. For N candidates and Q already-admitted depth zones, box association is O(N) and a standard-library median sort is O(Q log Q), with O(Q) extra memory [12]. A planning case of 5 candidates and at most 2,268 zones gives $N + Q \log_2 Q$ about 25,300; 100 times the candidates (500) gives about 25,800. These are values of a scaling expression, not counted instructions or measured milliseconds. Camera transport, detector inference, mask construction, full-footprint projection/visibility checks, and control are excluded from this bound and must be budgeted separately. The matched blank-image Mask R-CNN CPU rerun measured about 2.82 s median, excluding the rest of that pipeline; it demonstrates a timing gap rather than person accuracy or a prediction of Pi/Jetson speed.

Why this approach. Nearest-person following could choose a visitor (US-04), so it is rejected. ByteTrack and appearance-based diagnostics have also been explored: raw track IDs continued beyond the selected-target wrapper's loss, while OSNet did not establish sustained selected identity. Neither result permits automatic reacquisition. The proposed baseline therefore retains explicit selection, conservative box association, and a latched loss (US-02, US-04). IoU is an association aid, not proof that two boxes contain the same person. Real identity continuity and masks/depth remain release blockers.

Edge cases.

- *Empty input:* no candidates, no mask, or fewer than three valid depth points gives UNAVAILABLE with reason LOST, NO_MASK, or NO_DEPTH (I7).
- *Duplicates:* a repeated frame_id returns DUPLICATE and does not extend the old expiry; duplicate candidate IDs return INVALID_INPUT.
- *Ties:* two candidates within 0.15 IoU of each other, or overlapping by 0.50 or more, give AMBIGUOUS rather than a guess.
- *Out of order or stale:* an older frame can never replace a newer result; anything older than 200 ms gives STALE.
- *Permission change:* Stop, Privacy, or a new generation during processing means the result is thrown away, so a frame computed before Stop cannot restart following (I8).
- *Known gap:* two people swapping places in the same spot can fool a box tracker. We will test this case directly.

4 Build-versus-reuse decisions

The table separates existing experimental work from proposed integrations. The team-code license is undecided. Library maturity, base license, and intended fit have been reviewed; actual deployment performance, pinned versions, dependency notices, and model/dataset rights still need verification. The resident permission model should reuse the existing consent/following bridge, rather than create a second consent state machine.

Table 2. What the team builds and reuses

Piece	Build or reuse	Library or service	License	Reason (maturity, license, performance, fit)
Call outcome state machine and pairing rules	Build	None	Team code	Build the call-outcome and pairing rules for US-03; integrate the existing resident consent bridge for US-02. The server is not deployed.
Relational storage	Reuse	SQLite	Public domain [1]	Transactions and partial indexes fit one server; load tests, backups, and dependency notices remain necessary [2].
HTTP and WebSocket server	Reuse	FastAPI on Uvicorn	MIT / BSD-3-Clause [3,4]	Maintained, permissive Python stack; async I/O is available, but blocking database/auth work must not block heartbeats.
Request validation	Reuse	Pydantic (used by FastAPI)	MIT [17]	Set extra='forbid', strict scalar types, and finite/range validators; the defaults do not enforce this entire contract [21].
Sign-in and accounts	Reuse	Keycloak	Apache-2.0 [5]	Mature OIDC service candidate; avoids custom passwords. Introspection timing and deployment are unmeasured.
Token verification on the server	Reuse	Authlib	BSD-3-Clause [18]	OIDC/OAuth client candidate; verify asynchronous introspection and error mapping with the pinned release before integration.
Timestamps and UUIDs	Reuse	Python datetime and uuid	PSF [8]	Standard library, so nothing to install; parsing RFC 3339 by hand is a common source of bugs.
Selection, IoU matching, and loss latch	Build	NumPy for array math	Team code; NumPy BSD-3-Clause [19]	Project-specific rules for US-04; NumPy is an optional array tool, not a demonstrated latency guarantee.
Person detector and masks	Reuse	TorchVision Mask R-CNN (first candidate)	BSD-3-Clause code [6]; weight terms checked separately	Existing offline mask adapter; sampled CPU timings exceed 200 ms. SSDLite is a separate box-only baseline and cannot replace selected-instance masks.

Piece	Build or reuse	Library or service	License	Reason (maturity, license, performance, fit)
Calibrated RGB–ToF projection	Build / reuse prototype	Existing standard-library Python contract	Team code; PSF runtime [8]	Reuse synthetic geometry checks; actual calibration, visibility evidence, sensor data, and movement integration are absent.
Simulation	Reuse	MuJoCo	Apache-2.0 [7]	Existing simulation/training tool; bounded successes and failed stopping/latency cases do not establish transfer to hardware.
Caregiver iOS app	Build	Swift and SwiftUI	Apache-2.0 with runtime exception; Apple SDK terms [14,15]	Native iOS UI fits the requested companion; build on Austin's prior SwiftUI work. Real calling, accessibility, and background behavior remain untested.

5 API contract

This section specifies I1 and I2 on the call server and I7/I8 for person tracking. Rules that apply everywhere: JSON field names are case-sensitive and unknown fields are rejected. UUIDs are canonical 36-character strings, and timestamps are RFC 3339 UTC strings such as 2026-10-02T14:30:00Z. Every caregiver request carries its token in an Authorization: Bearer <token> header (1–4096 characters), never in the body, and the server never stores or logs it. Every HTTP error contains an error object with code and message, both text, and no success fields. Map framework validation failures to the specified 400 response; do not leak tokens or raw input in error logs. A pairing makes an account eligible to call; it is not the resident's permission.

Table 3. API summary

Endpoint or method	Inputs	Output	Errors (code: when)
POST /v1/call-commands (I1)	Bearer token; robot_id, action, optional request_id, call_id for end	CallStatus; 202 for a new ringing call, 200 otherwise	400: bad input; 401: bad or expired token; 403: not paired; 404: unknown robot or call; 409: active call or retry ID conflict; 503: sign-in or database down
GET /v1/calls/{call_id} (I1)	Bearer token; call_id in path	CallStatus, 200	400: bad UUID; 401; 403; 404; 503
WSS /v1/calls/{call_id}/updates (I1)	Bearer token on connect; call_id in path	CallStatus on every change	Close 1008: bad or unauthorized token; close 1011: server failure
WSS /v1/robots/{robot_id}/session (I2)	Robot token; heartbeat and event frames	Invite and cancel frames	Error frame INVALID_MESSAGE or STALE_CALL; close 1008: bad robot identity; close 1011: storage failure

Endpoint or method	Inputs	Output	Errors (code: when)
trackSelected (I7, I8)	Frame, candidates, depth, gates, prior state	TrackRecord	TrackError INVALID_INPUT, NO_SELECTION, or DUPLICATE

CallStatus is {call_id: UUID, status: call-status enum, last_seen: UTC timestamp or null}.

Call server: I1

POST /v1/call-commands. The body is a JSON object with these fields:

Input	Type, requirement, and valid values
robot_id	Required text, 1–64 characters, letters, numbers, hyphen, or underscore; an enrolled robot.
action	Required enum: start or end.
request_id	Optional UUID or null for start; when supplied, unique per caregiver. It must be omitted or null for end.
call_id	Required UUID for end; omitted or null for start. It must identify a call for the given robot and authenticated caregiver's pairing. Ending a terminal attempt returns its existing result.

Success, HTTP 202 or 200: a CallStatus object. A new start to an online robot returns 202 with ringing; a start to an offline robot returns 200 with offline. A retry or an end returns 200 with the current or final status. ringing does not mean the resident accepted; only connected does.

Errors: 400 INVALID_REQUEST for invalid types, ranges, or conditional fields; 401 UNAUTHENTICATED for an invalid/expired token; 403 NOT_PAIRED for a valid account without an active pairing; 404 NOT_FOUND for an unknown robot or call; 409 ACTIVE_CALL or REQUEST_CONFLICT for a second active call or a retry ID reused with different input; 503 SERVICE_UNAVAILABLE if the identity service or database is unavailable. The caller may retry a 503, preferably with the same request ID; it must not treat a rejected request as a connected call.

GET /v1/calls/{call_id}. Input is a required UUID path parameter plus the Bearer header. The caller must own the active pairing for the call. HTTP 200 returns a CallStatus. Errors: 400 INVALID_REQUEST for a bad UUID, 401 UNAUTHENTICATED, 403 NOT_PAIRED, 404 NOT_FOUND, and 503 SERVICE_UNAVAILABLE.

WSS /v1/calls/{call_id}/updates. The app opens this socket with the same Bearer header. After the token and pairing checks, the server sends a CallStatus right away and again whenever the status or last-seen time changes; the client sends nothing. The server closes with code 1008 for a missing, invalid, or unauthorized token and 1011 for a sign-in or storage failure. If the socket drops, the app shows that it lost contact and can call GET after reconnecting; a dropped socket never means the call connected.

Example I1 request and response:

POST /v1/call-commands
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{"robot_id": "home-robot-01", "action": "start",
"request_id": "e5c7cb31-343c-4fb8-a632-054e84465d24"}

HTTP/1.1 202 Accepted
Content-Type: application/json

{"call_id": "f34f4e06-d38c-4dbd-a9b1-d11650d06599",
"status": "ringing", "last_seen": "2026-10-02T14:30:00Z"}

HTTP/1.1 409 Conflict (new request_id; another call is active)
{"error": {"code": "ACTIVE_CALL",
"message": "This robot already has an active call."}}

Robot session: I2

WSS /v1/robots/{robot_id}/session. The path has the same robot ID rules as I1. An authenticated robot service token in the Bearer header is required; its subject must match the enrolled robot. Robot-to-server JSON frames are a heartbeat with type:"heartbeat" and integer sequence, sent every second, or an event with type:"event", UUID call_id, and event. Sequence ranges from zero to 2^{63}-1; event is ACCEPTED, DECLINED, MEDIA_CONNECTED, MEDIA_FAILED, or ENDED. Server-to-robot JSON frames are an invite with type:"invite", UUID call_id, text caller_name, and timestamp expires_utc; or a cancel with type:"cancel", UUID call_id, and reason. Reason is ENDED, NO_ANSWER, OFFLINE, or FAILED. Names are 1–80 characters. Sequence is scoped to the current authenticated session: opening a new session retires the old socket and clears last_sequence; subsequent heartbeats must increase it. Old-session messages cannot refresh last_seen_utc or revive a call.

A malformed or invalid event gets {type:"error", code:"INVALID_MESSAGE" or "STALE_CALL"}; it cannot change a terminal call. The server closes with WebSocket code 1008 for a bad robot identity or repeated invalid messages and 1011 for an internal storage failure. When the session drops, the server marks the robot offline after 5 seconds, and the robot ends any call in progress (D0, I2).

Person tracking method: I7 and I8

trackSelected(input) -> TrackRecord or TrackError. This is an in-process function call on the robot, not an HTTP endpoint. Every field below is required unless marked optional. Time values are integer milliseconds in one robot monotonic clock domain, from 0 to 2^{63}-1.

Input	Type, requirement, and valid values
Selection and gates	target_id: text, 1–64 characters; call_accepted and follow_allowed: booleans; generation: nonnegative integer.
Time and frame	now_ms, frame_id, capture_ms: nonnegative integers. Image width and height: positive integers, each at most 1920, with width times height at most 2,073,600 pixels.
Candidates	Array of 0–500 records: candidate_id is unique text of 1–64 characters; box is [left, top, right, bottom] in integer pixel coordinates with positive area; left/top are inclusive, right/bottom exclusive, all within image bounds. confidence is a finite number in [0,1]. mask is an optional image-sized boolean array.
Depth projection	Required record: target_id, candidate_id (text, 1–64 characters), frame_id, generation, and capture_ms (nonnegative integers); these must match the selected observation. status: candidate/not_observed/uncertain; source: synthetic/offline/hardware. points: 0–2,268 already-admitted records with finite pixel x, y, positive finite optical_z_m, and z_interval_m: two ordered finite positive bounds enclosing that depth.
Calibration	fx_px: finite number greater than zero and at most 100,000; cx_px: finite number from zero inclusive to width exclusive, in pixels for the rectified RGB image. calibration_id: text, 1–64 characters, naming the validated revision for this image mode. The upstream adapter also needs measured zone rays, intrinsics/extrinsics, bounded reprojection/range error, synchronized capture times, and per-zone clear visibility; its validated calibration revision must match the frame mode. Nominal field of view is insufficient.
Selection state	Optional selected_candidate_id: text or null, required for a new explicit selection. Optional prior: null or a record with generation, last_frame_id (nonnegative integers), last_box (four in-bounds pixel integers), and loss_latched (boolean).

New selection requires a fresh resident Follow action, a selected candidate, and either no prior state or a generation greater than the prior generation. A latched loss cannot be cleared by later frames alone. The tracker atomically updates its per-target prior state when processing a frame; the next call receives that state. Duplicate frames do not update it.

TrackRecord: fields are target_id: text, tracking_status: valid/unavailable, range_m and bearing_deg: finite number or null, capture_ms and expires_ms: integer milliseconds, reason: enum, range_frame: constant rgb_optical_z, and motion_authorized: constant false. Range is median RGB optical-axis Z in metres; bearing is in degrees, positive right. It is not a robot-frame separation or clearance. Valid means an offline candidate with reason OK, expiring by capture + 200 ms; it never grants movement. Unavailable has null range/bearing and reason CONSENT_OFF, STALE, AMBIGUOUS, LOST, NO_MASK, NO_DEPTH, or BAD_CALIBRATION. Missing calibration gives BAD_CALIBRATION; not_observed, uncertain, or missing visibility gives NO_DEPTH. Stop or a generation change invalidates earlier output immediately. A qualified movement interface requires separate base-frame conversion, identity/clearance checks, and physical validation in D2; it cannot reinterpret this offline range as a robot-frame command.

TrackError: {code:enum, message:text}. INVALID_INPUT covers missing fields, invalid bounds or

duplicate IDs; NO_SELECTION covers a missing required resident selection; DUPLICATE covers an already processed frame. INVALID_INPUT and NO_SELECTION make the caller request a stop. DUPLICATE must not extend the prior record's expiry; the watchdog requests a stop when freshness expires, even if no new method call arrives. The proposed integration serializes permission-generation checks with publication; this atomic cross-component behavior still needs testing. Offline bridge tests alone do not establish it on hardware.

Versioning. The HTTP and WebSocket APIs are versioned by the /v1 path prefix and the tracking method by a version constant in its module; removing or renaming a field, changing a type, unit, required input, status meaning, or error behavior is a breaking change that requires /v2, while adding an optional output field is not breaking (adding an input field is, because v1 rejects unknown inputs).

6 Technology choices with justification

Each paragraph covers team skill fit, licensing, community support, performance, and cost/hosting. These are proposed selections for review; unmeasured capacity, prices, and hosting allowances are not established facts.

Database: SQLite. *Skill:* Python is already used for simulation and perception; the team will still need to implement the SQL schema and migrations. *License:* SQLite's core is public domain, with dependency and deployment notices checked separately [1]. *Community:* official documentation covers transactions, indexing, and deployment [2]. *Performance:* one server is a reasonable starting design, but neither one-robot latency nor 100-robot write contention has been benchmarked. *Cost and hosting:* no database license fee, but durable storage, backups, and administration require an approved host. *Alternative:* PostgreSQL adds an independently operated database service and is the candidate if measured write contention or multi-host deployment warrants it.

Backend: Python with FastAPI on Uvicorn. *Skill:* existing Python experiments provide relevant experience; JavaScript screen mocks are a separate existing code path. *License:* Python PSF, FastAPI MIT, and Uvicorn BSD-3-Clause [3,4,8]. *Community:* both server projects have maintained official documentation. *Performance:* asynchronous I/O can overlap network waits, provided database and authentication work do not block the event loop; a worker count or two-second response guarantee cannot be chosen without measurements. *Cost and hosting:* the software has no license fee; compute, monitoring, and integration still cost resources. *Alternative:* Node.js/Express would use the team's JavaScript experience, but Python keeps the initial server aligned with the simulation/perception tooling.

Caregiver front end: iOS app in Swift and SwiftUI. *Skill:* Austin's prior Swift/SwiftUI app work supports the choice, with Gavin proposed to design the screens. The robot companion itself is not implemented. *License:* Swift is Apache-2.0 with a runtime exception; Apple SDK and team-code terms are separate [14,15]. *Community:* Apple documentation and Swift forums support development; VoiceOver and text sizing still need app-specific tests [16]. *Performance:* foreground call requests and status updates are the initial target. Incoming background calls would need a separately designed APNs/PushKit/CallKit path; no locked-phone ringing or hardware-encoder result has been demonstrated [20]. *Cost and hosting:* a free developer account supports own-device testing; wider distribution may need the $99/year

program, plus a Mac/iPhone and backend hosting [15]. *Alternative:* a browser companion remains a cross-platform option; the requested iOS direction motivates native development, not an untested claim that it solves notification reliability.

Identity and messaging: Keycloak, HTTPS, and secure WebSocket. *Skill:* Python integration is plausible, but production Keycloak operation is not recorded; Austin is the proposed integration owner. *License:* Keycloak Apache-2.0, with Authlib BSD-3-Clause as a client candidate [5,18]. *Community:* official OIDC documentation supports integration [9]. *Performance:* token introspection adds a network operation; its delay, resource footprint, and effect on heartbeats are unmeasured. *Cost and hosting:* no software license fee, but identity storage and operations may need a separate service or a larger host; resource sizing remains to be measured. *Alternative:* managed identity such as Firebase Authentication could reduce administration, but introduces provider terms and costs to evaluate; no provider is deployed.

On-robot perception and hardware: Python/TorchVision, host to be selected. *Skill:* existing offline detector and projection work supports this software direction. *License:* TorchVision/PyTorch base code and bundled dependencies have separate notices; dataset/weight rights remain to be resolved [6,13]. *Community:* official TorchVision, ST, and Raspberry Pi documentation provide supported integration references [6,10,22]. *Performance:* SSDLite is box-only; the measured Mask R-CNN CPU baseline misses 200 ms. Neither Pi 5 nor Jetson performance has been measured. The purchased/shipped VL53L9CXV0VE/1 is a bare depth component, not an RGB-D USB device; carrier, host, real frames, calibration, and power integration remain unverified. A Pi 5 candidate has two shared camera/display ports, so RGB CSI + ToF MIPI + DSI display needs a different verified route [10]. *Cost and hosting:* board variant, carrier, camera, display, storage, power and integration prices need quotes; no replacement build budget is approved. *Alternative:* a Jetson is an evaluation candidate if compute needs justify it, without a promised speedup or automatic hardware switch.

Hosting: proposed Ubuntu Server VM. *Skill:* public-server operating experience is not documented; Austin is the proposed deployment owner. *License:* Ubuntu comprises packages with distinct licenses [11]. *Community:* official platform and server-project deployment documentation are available. *Performance:* test the complete server, identity service, and any media relay against heartbeat and call targets before choosing RAM or worker counts. *Cost and hosting:* provider, VM size, persistent disk, backups, TLS operation, relay traffic, and monthly allowance remain unapproved. *Alternative:* a laptop can support local integration, but remote reachability and availability require separate validation.

Source notes

[1] SQLite copyright and public-domain core; [2] SQLite deployment and single-writer guidance; [3] FastAPI project and license; [4] Uvicorn project and license; [5] Keycloak project and license; [6] TorchVision project, code license, and weight notice; [7] MuJoCo project and license; [8] Python license; [9] Keycloak OIDC endpoints; [10] Raspberry Pi connector documentation; [11] Ubuntu package and license policy; [12] Python standard-library median; [13] PyTorch code and bundled dependency licenses; [14] Swift license; [15] Apple developer tools, testing, and distribution; [16] SwiftUI accessibility fundamentals; [17] Pydantic project and license; [18] Authlib project and license; [19] NumPy license; [20] Apple incoming VoIP notification requirements; [21] Pydantic validation configuration; [22] ST VL53L9CX component specifications.